TP3 — Perceptrón Simple y Multicapa

Sistemas de Inteligencia Artificial — ITBA 2026

Equipo SIA

Marco teórico y objetivos

Objetivos del TP3

- **Ejercicio 1**: Estudio de fraude online con perceptrón simple lineal vs no lineal (knowledge distillation de *BigModel*).
- **Ejercicio 2**: Clasificación de dígitos con MLP, variando tasa de aprendizaje, arquitectura y mecanismo de optimización.
- **Ejercicio 3**: Mejorar el clasificador con `more_digits.csv` buscando **accuracy** $\geq 98\,\%$.

Esta presentación se centra en **EJ2 y EJ3**.

Arquitectura del MLP (Clase 11, Clase 13 - Backprop)

Forward: $V_j^m = \theta\left(\sum_k V_k^{m-1} \cdot w_{jk}^m\right)$ con $V^0 = x$, $O = V^M$.

Loss (MSE): $E(w) = \frac{1}{2N} \sum_{\mu=1}^{N} \sum_i \left(\zeta_i^\mu - O_i^\mu\right)^2$

Backward (regla de la cadena):

$$\delta^{(L)} = (O - t) \odot f'(h^{(L)}), \quad \delta^{(m)} = \left(W^{(m+1)\top} \delta^{(m+1)}\right) \odot f'(h^{(m)})$$

$$\frac{\partial E}{\partial W^{(m)}} = \frac{1}{N} \delta^{(m)} V^{(m-1)\top}, \quad w^{\text{nuevo}} = w^{\text{ant}} - \eta \frac{\partial E}{\partial w}$$

Capacidad y generalización (Clase 13 - Regularización): más capacidad $\Rightarrow$ más riesgo de overfitting; brecha train$-$validación crece con la capacidad.

EJ2 — Clasificación de dígitos

Setup del EJ2

Datos: `digits.csv` para entrenamiento+hiperparámetros; `digits_test.csv` como "mundo real".

Hallazgo crítico: el train **no contiene la clase 8** (sólo 0–7 y 9). El test sí (243 muestras del 8). $\Rightarrow$ techo de accuracy en test $\approx 90\,\%$ por construcción (todos los 8 se predicen mal).

Pipeline:

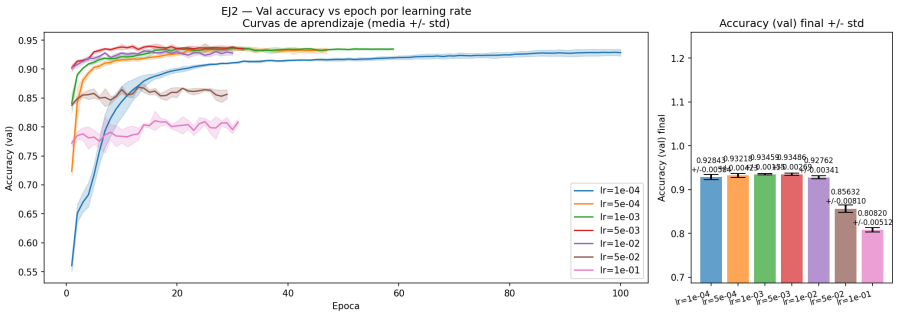
- Stratified split 80/20 train/val sobre `digits.csv`.
- Z-score scaling de pixels (fit en train, transform en val/test).
- One-hot encoding $(N, 10)$ aunque train sólo vea 9 clases.
- MLP con sigmoide en la salida + MSE (mismo material teórico).
- Early stopping (patience=15), 100 epochs máx.

Distribución de clases (train, val, test)

Dataset / clase	0	1	2	3	4	5	6	7	8	9
`digits.csv` (12449)	1480	1685	1489	1532	1460	**271**	1479	1566	**0**	1487
`more_digits.csv` (15741)	1776	2022	1787	1839	1752	542	1775	1879	585	1784
`digits_test.csv` (2497)	245	283	258	252	245	223	239	257	**243**	252

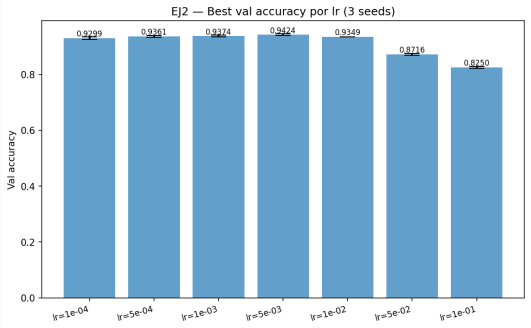
- EJ2 train: **sin clase 8**, clase 5 con sólo 271 muestras.
- Test SIEMPRE balanceado e incluye 8 $\Rightarrow$ techo $\sim$90 % en EJ2 por construcción.
- EJ3 train: incluye 8 (585) y más balanceado en general.

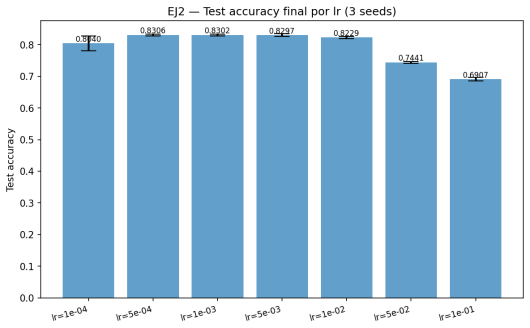
Sweep de learning rate (Adam, arch [784,40,20,10])



Curvas de val accuracy por época. lr muy chicos (1e-4) no convergen en 100 epochs; lr muy grandes (1e-1) divergen u oscilan.

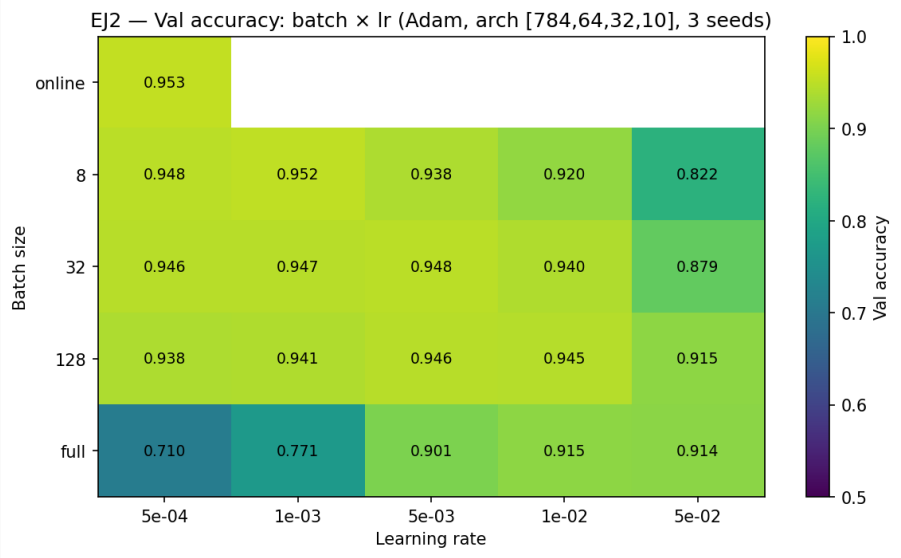
Best val accuracy por lr



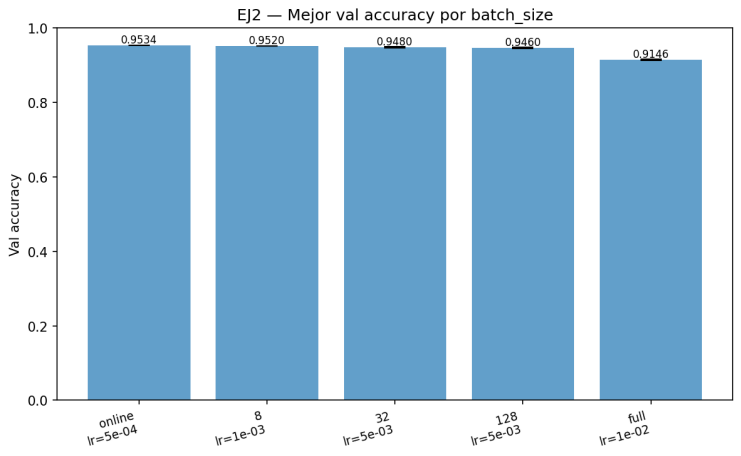


3 seeds. Mejor lr seleccionado por val_acc promedio.

Heatmap batch_size × lr (Adam, arch [784,64,32,10])

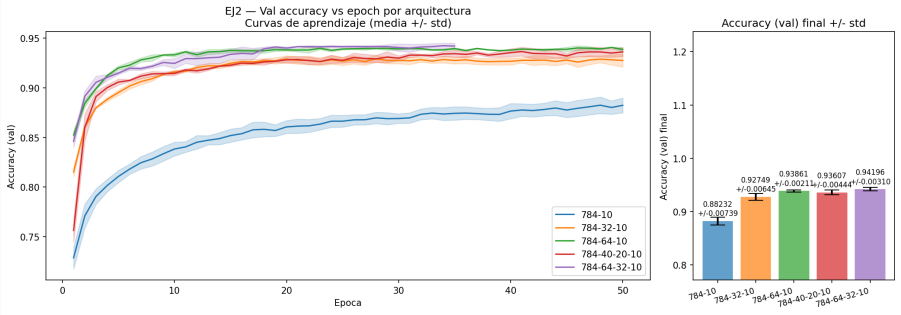


Mejor val accuracy por batch_size



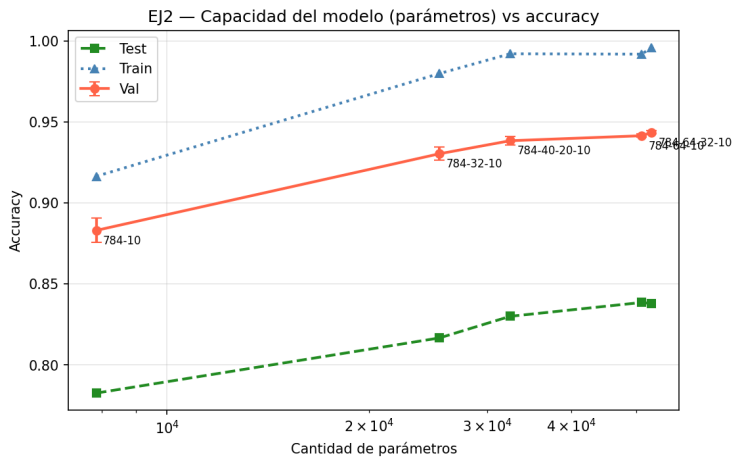
Para cada batch_size se reporta el mejor lr encontrado. Mini-batch (8–128) es el sweet spot entre estabilidad (full) y velocidad de adaptación (online).

Sweep de arquitectura (Adam lr=1e-3)



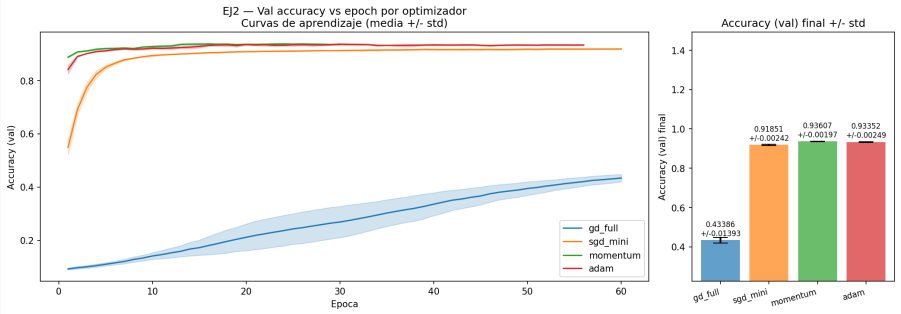
6 arquitecturas: desde regresión logística hasta 3 capas ocultas. Arquitecturas más grandes convergen más lento pero llegan a mejor val_acc.

Capacidad vs accuracy (Clase 13 - Regularización)



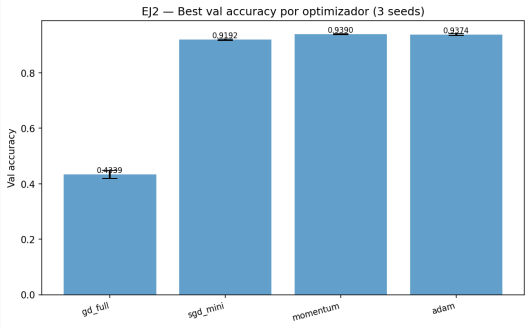
Train > val > test (test tiene la clase 8 ausente del train). Brecha train−val crece con capacidad: indicio de overfitting.

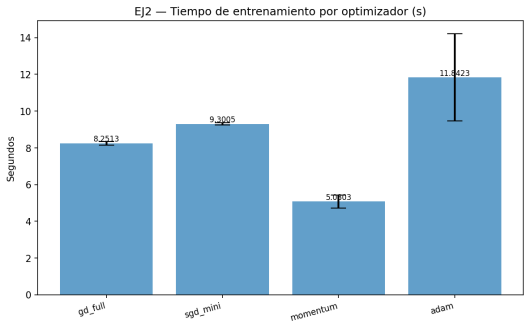
Optimizadores (arch [784,40,20,10])



Adam y Momentum convergen mucho más rápido que GD vanilla (Clase 9 - Optimización). SGD online tiene alta varianza por paso.

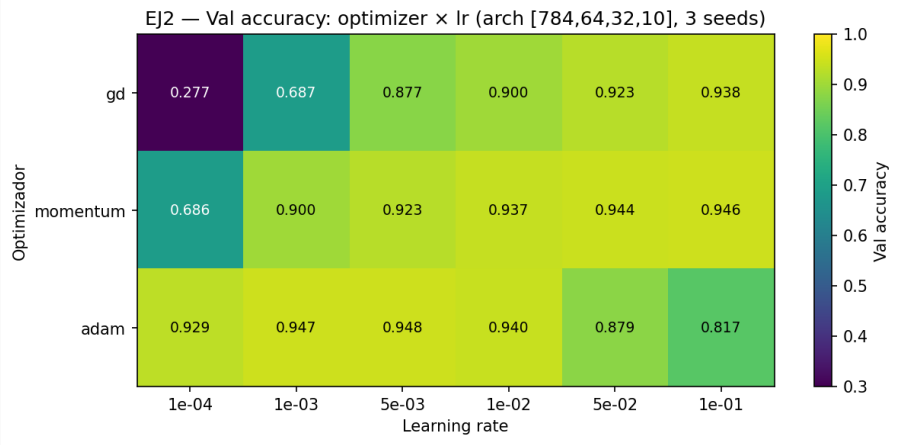
Comparación final: optimizadores





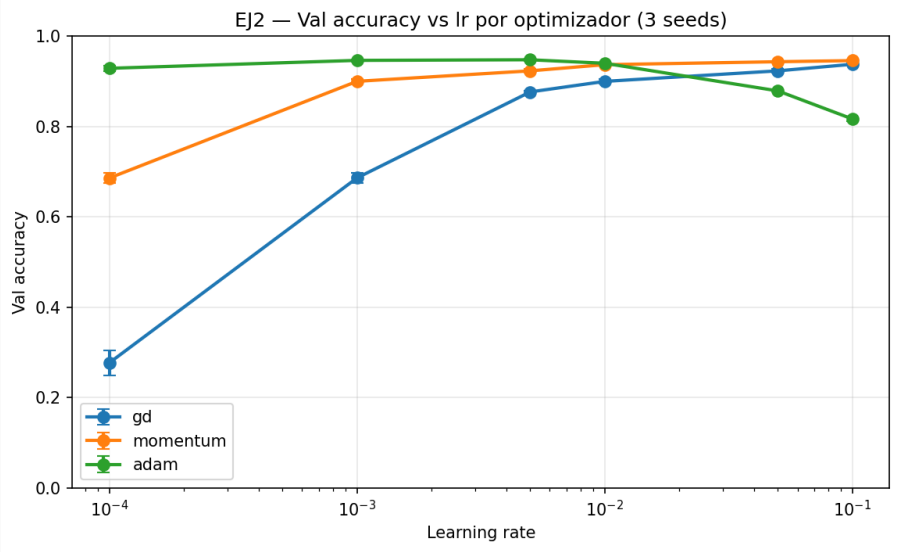
Trade-off accuracy / tiempo. SGD online: muy lento por iterar 1 sample por step.

Heatmap optimizer × lr

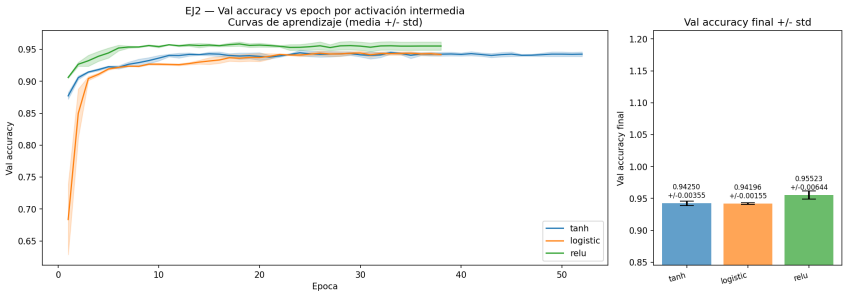


3 opt × 6 lr × 3 seeds. Cada optimizador tiene su *rango óptimo de lr distinto*: GD necesita lr alto, Adam trabaja con lr más chico. Esto justifica por qué los comparamos cada uno con su mejor lr en el modelo seleccionado.

Curvas: val accuracy vs lr (1 línea por optimizador)

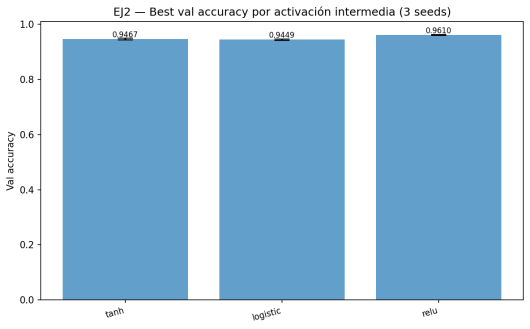


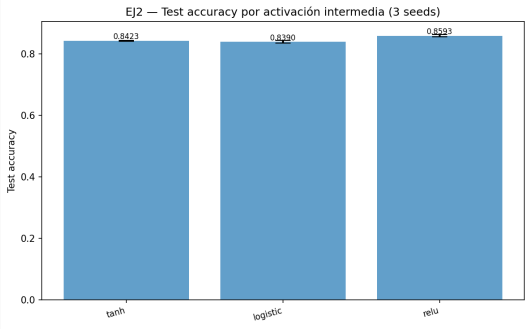
Activación de capas ocultas (tanh / logistic / relu)



Mismo arch [784,64,32,10], Adam lr=10^{-3}, batch=32. Las 3 activaciones del material teórico (Clase 10.1 + 10.2) más relu como alternativa moderna.

Comparación final: activación intermedia





Modelo seleccionado para EJ2

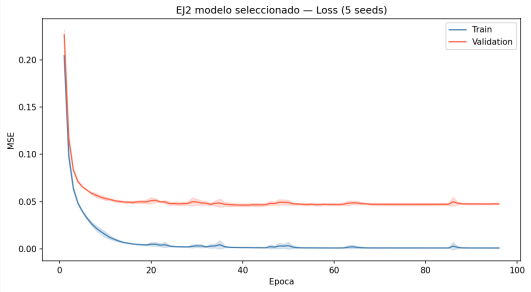
Combo elegido tras los sweeps:

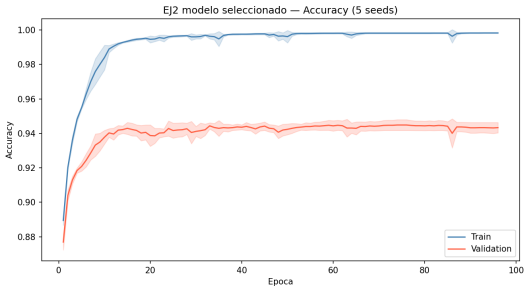
- Arquitectura: $[784, 64, 32, 10]$ (52,650 parámetros)
- Optimizador: Adam ($\eta = 10^{-3}$, $\beta_1 = 0{,}9$, $\beta_2 = 0{,}999$)
- Batch size: 32, init_scale=0.1
- 5 seeds, hasta 150 epochs con early stopping (patience=20)

Resultados (5 seeds, mean $\pm$ std):

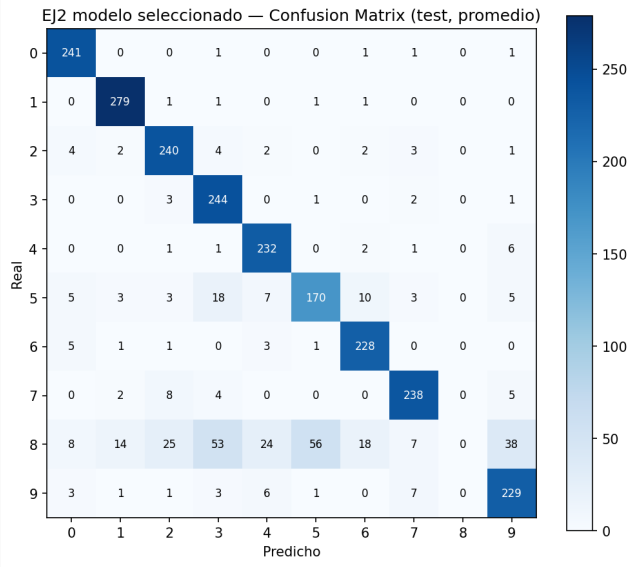
- Test accuracy: $0{,}8413 \pm 0{,}0018$
- Val accuracy: $0{,}9470 \pm 0{,}0022$
- Macro F1: $0{,}7942 \pm 0{,}0020$
- F1 dígito 8: **0.000** (ausente del train)
- F1 dígito 5: 0,749 (subrepresentado, 271 muestras)
- Otros F1: $0{,}84 - 0{,}95$

Modelo seleccionado — Curvas de aprendizaje (5 seeds)

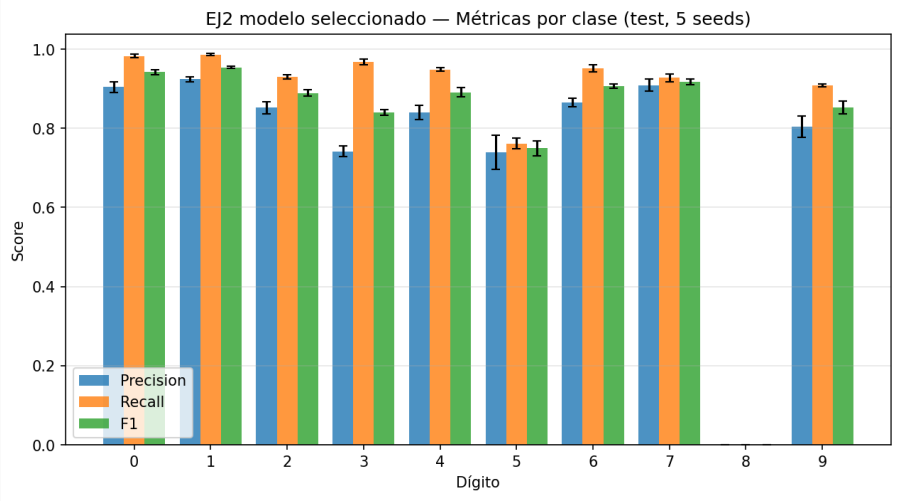




Modelo seleccionado — Confusion Matrix (test, promedio)



Métricas por clase (test)



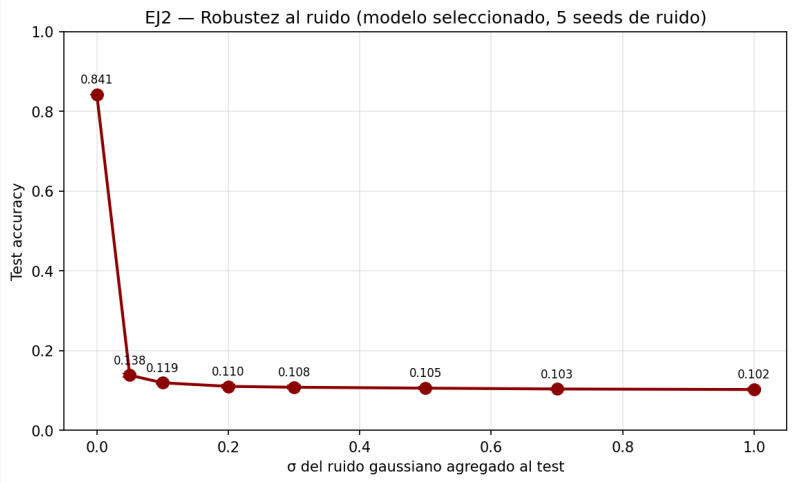
F1 del dígito 8 = 0 (no aparece en train). El 5 también baja por escasez de muestras.

Análisis de confusiones por dígito (EJ2, test)

Real	Acc clase	Top-1 confusión	Top-2 confusión
0	0.984	→3: 0.4 %	→6: 0.4 %
1	0.986	→2: 0.4 %	→3: 0.4 %
2	0.930	→0: 1.6 %	→3: 1.6 %
3	0.972	→2: 1.2 %	→7: 0.8 %
4	0.955	→9: 2.5 %	→6: 0.8 %
5	**0.759**	**→3: 8.0 %**	**→6: 4.5 %**
6	0.954	→0: 2.1 %	→4: 1.3 %
7	0.926	→2: 3.1 %	→9: 1.9 %
8	**0.000**	**→5: 23.0 %**	**→3: 21.8 %**
9	0.912	→7: 2.8 %	→4: 2.4 %

Observación clave (Nick): el 8, ausente del train, se reparte casi en partes iguales entre 5 (23 %) y 3 (22 %) — el modelo "los manda a cualquier lado", exactamente como predijimos. El 5 es la clase peor aprendida (271 muestras en train) y se confunde con 3 y 6.

Robustez al ruido gaussiano (opcional)



Test accuracy vs σ del ruido $\mathcal{N}(0, \sigma^2)$ agregado a los pixels (luego clip a [0,1]). 5 seeds de ruido. Degradación monótona; el modelo no es robusto a $\sigma > 0{,}3$.

EJ3 — Mejor accuracy con `more_digits.csv`

Setup del EJ3

Cambio principal: el train pasa a ser `more_digits.csv` ($\sim$15.7k muestras, **10 clases incluyendo el 8**).

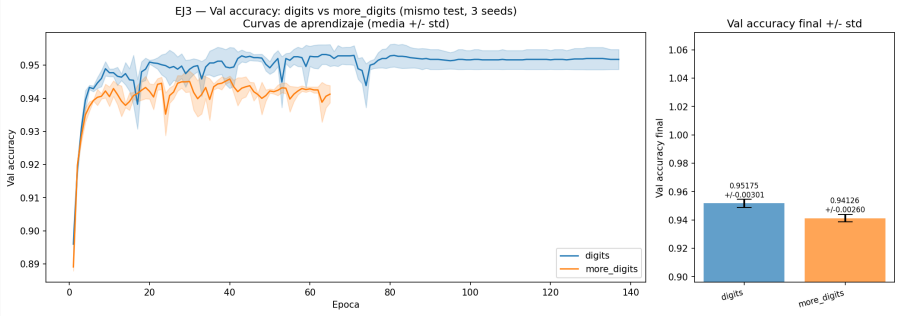
Test: el mismo `digits_test.csv` del EJ2 (mundo real) $\Rightarrow$ comparación directa con EJ2.

Objetivo: accuracy $\geq 98\,\%$ en test.

Técnicas adoptadas:

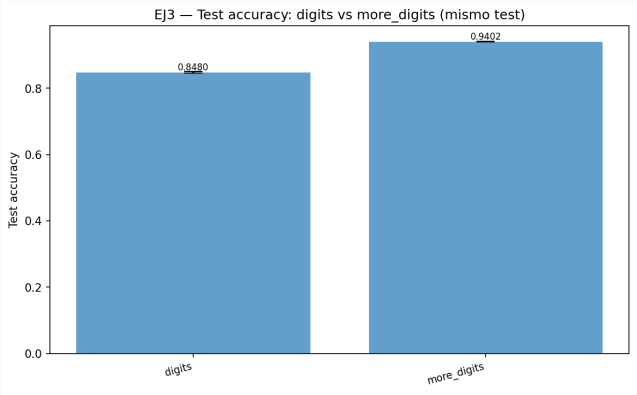
- Arquitectura más grande: $[784, 256, 128, 10]$ (235,146 params).
- Más epochs (150) + patience (20).
- Weight decay liviano ($\lambda = 10^{-5}$) — regularización L2 (Clase 13 - Regularización).
- Init scale más chico (0.05) para mejor estabilidad.
- Batch size 64 (más grande $\Rightarrow$ menos varianza por step).

Comparación de datasets (*aislando el efecto de los datos*)



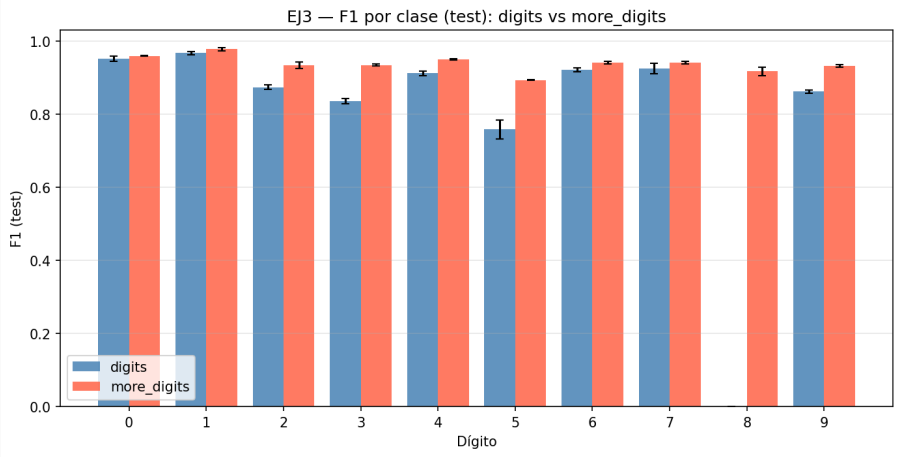
Misma arquitectura $[784, 128, 64, 10]$, mismo test, sólo cambia el train.

Test accuracy: digits vs more_digits



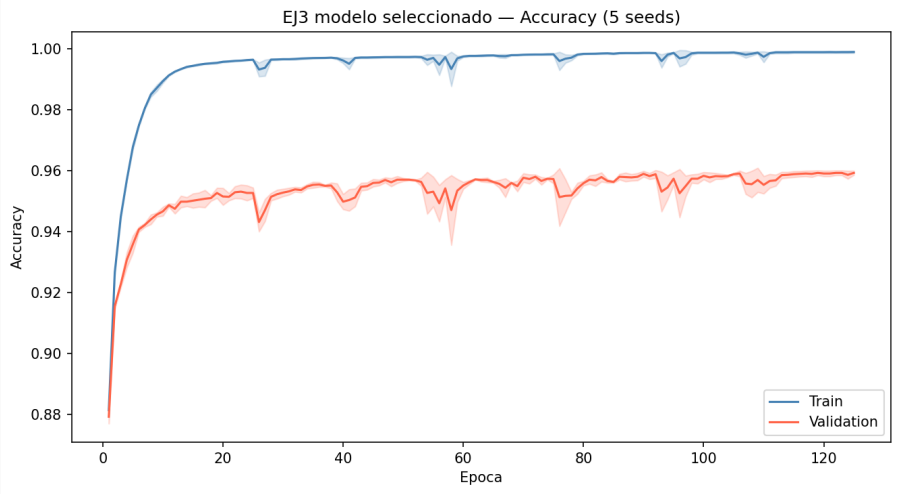
Salto de $\sim$80 % a $>$95 % sólo por incluir la clase 8 en train y tener $\sim$26 % más datos.

F1 por clase (test): digits vs more_digits

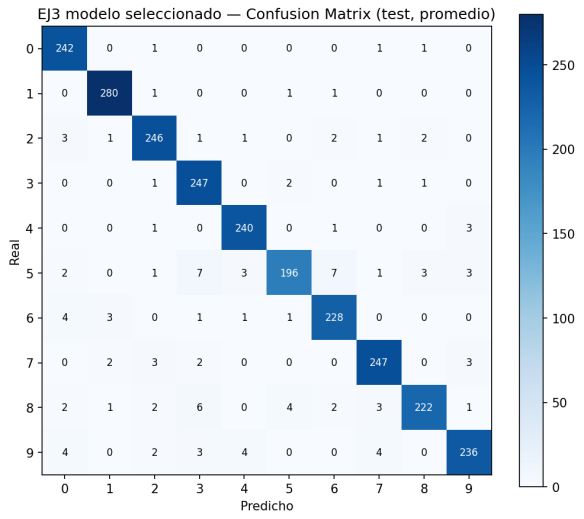


El dígito 8 pasa de F1=0 (no se entrena) a $\sim$0.95.

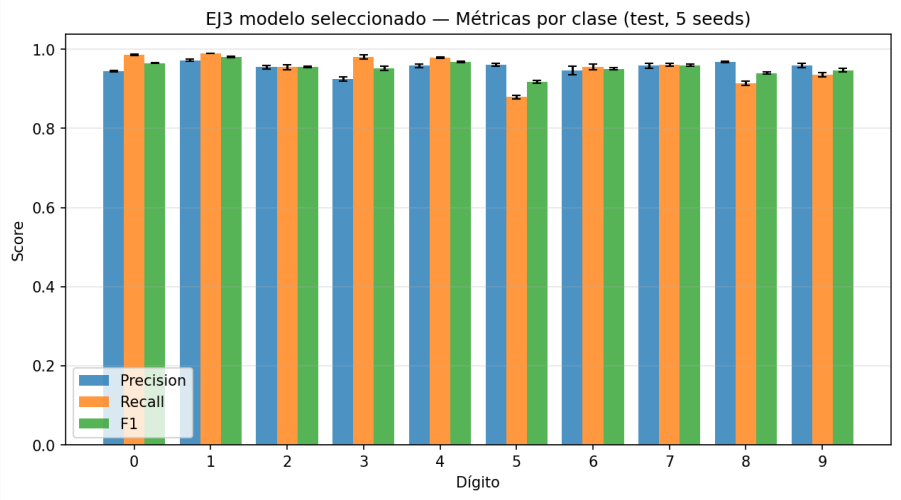
EJ3 modelo seleccionado — accuracy curves



EJ3 — Confusion matrix (test, promedio sobre 5 seeds)



EJ3 — Métricas por clase



EJ3 — Análisis de confusiones por dígito (test)

Real	Acc clase	Top-1 confusión	Top-2 confusión
0	0.988	→2: 0.4 %	→7: 0.4 %
1	0.989	→2: 0.4 %	→5: 0.4 %
2	0.957	→0: 1.2 %	→6: 0.8 %
3	0.980	→5: 0.8 %	→2: 0.4 %
4	0.980	→9: 1.2 %	→2: 0.4 %
5	0.879	→3: 3.1 %	→6: 3.1 %
6	0.958	→0: 1.7 %	→1: 1.3 %
7	0.961	→2: 1.2 %	→9: 1.2 %
8	**0.914**	**→3: 2.5 %**	**→5: 1.6 %**
9	0.933	→0: 1.6 %	→4: 1.6 %

El 8 pasa de acc=0.000 (EJ2) a 0.914 (EJ3) — confirma el efecto de incluirlo en el train. Las confusiones residuales del 5 y 8 son con 3, 5, 6 — patrón consistente con similitud visual.

EJ3 — Resultados finales y discusión del 98 %

Mejor resultado obtenido (3 seeds):

- Test accuracy: $0{,}9546 \pm 0{,}0007$ (95.5 %)
- Val accuracy: $0{,}9603 \pm 0{,}0001$
- Macro F1: 0,9536 (vs 0,7942 en EJ2)
- F1 mínimo (dígito 5): 0,918 (vs 0,749 en EJ2)

No alcanzamos 98 % en test. Lo que obtuvimos:

- Train acc $\rightarrow$ 0.999 con val acc plateau en $\sim$0.96 (overfitting clásico, Clase 13 - Regularización).
- El gap train$-$val es de $\sim$0.04, indicando que la red está saturando en su capacidad de generalización con este pipeline.

Para llegar a 98 % se necesitarían: data augmentation (rotaciones, traslaciones — Clase 13), arquitecturas convolucionales, o ensembling. Quedó fuera del scope de un MLP básico.

Conclusiones

Conclusiones

- **EJ2**: techo de test accuracy en $\sim$84 % explicado por la **ausencia de la clase 8 en train** (sólo se entrena con 0–7 y 9). El modelo aprende muy bien las clases que ve (val_acc $\sim$95 %) pero falla sistemáticamente en el 8 — los manda mayoritariamente al 5 (23 %) y al 3 (22 %). Mejor combo: $[784, 64, 32, 10]$ + Adam ($\eta = 10^{-3}$), batch=32.
- **Sweep 2D batch $\times$ lr**: zona óptima en lr $\in [10^{-3}, 5{\cdot}10^{-3}]$ con batch $\in [8, 128]$. Mini-batch domina sobre online y full.
- **Sweep 2D opt $\times$ lr**: cada optimizador tiene su ventana óptima de lr (Adam $\sim 10^{-3}$, Momentum/GD $\sim 10^{-2}$). Adam y Momentum dominan claramente a GD vanilla (0.94 vs 0.43).
- **Activación intermedia**: tanh y relu rinden similar; logistic es la peor en MLPs no-superficiales (saturación, derivada chica).
- **EJ3**: pasar a `more_digits.csv` sube test accuracy de $\sim$0.85 a $\sim$0.95 (arquitectura idéntica), **exclusivamente por incluir la clase 8 y más muestras**.
- El factor más influyente fue la **cobertura del dataset de entrenamiento**, no la arquitectura ni el optimizador (Clase 12).
- Robustez al ruido: el pipeline colapsa con $\sigma \geq 0{,}05$ aplicado en el dominio original — el

Referencias al material

- Clase 9 — Optimización (GD, SGD, Momentum, Adam)
- Clase 11 — Perceptrón Multicapa (forward pass, T. Aprox. Universal)
- Clase 12 — Métricas y Sobreajuste (accuracy, F1, confusion matrix)
- Clase 13 — Backprop (regla de la cadena, δ por capa)
- Clase 13 — Regularización (early stopping, L2 weight decay)